

Day 2: Bare Metal vs Zephyr Hello World — LED Blink

Platform: nRF52840 | RTOS: Zephyr RTOS

Learning Objectives

Bare Metal Startup Sequence

Before `main()` runs, the CPU executes startup code:

1. **Reset handler** is called (address stored in vector table at 0x00000004)
2. Zero-initializes `.bss` section (global uninitialized variables)
3. Copies `.data` section from Flash to RAM (initialized globals)
4. Calls C++ static constructors
5. Calls `main()`

In Zephyr, this is handled by `arch/arm/core/cortex_m/reset.S` — you don't write it yourself.

Zephyr Application Lifecycle

A Zephyr application starts in `main()`. After `main()` returns, the **idle thread** takes over, putting the CPU into a low-power wait state. Zephyr's scheduler runs other threads (if any) between `k_msleep()` calls in your code.

GPIO API: `gpio_pin_configure_dt` & `gpio_pin_toggle_dt`

Zephyr's GPIO API is device-tree-aware:

```
// Get pin spec from DTS (compile-time, no hardcoded pin numbers)
static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios);
```

```

gpio_is_ready_dt(&led); // Check driver is ready
gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE); // Set as output, initially active
gpio_pin_toggle_dt(&led); // Toggle the pin
gpio_pin_set_dt(&led, 1); // Set HIGH
gpio_pin_set_dt(&led, 0); // Set LOW

```

Devicetree Aliases (led0, led1...)

The nRF52840DK board DTS defines:

```

aliases { led0 = &led0; led1 = &led1; /* ... */ };
leds { led0: led_0 { gpios = <&gpio0 13 GPIO_ACTIVE_LOW>; }; };

```

Your code uses `DT_ALIAS(led0)` — if you port to a different board, only the DTS changes, not your C++ code.

struct gpio_dt_spec

`gpio_dt_spec` is a struct holding the GPIO device pointer, pin number, and flags — all extracted from DTS at compile time. It's the type-safe way to reference a GPIO in Zephyr.

k_msleep for Cooperative Yielding

`k_msleep(500)` suspends the current thread for 500ms and yields the CPU to other threads. This is **not** a busy-wait — the CPU enters an idle/sleep state between ticks, saving power.

Practice Tasks

1. Blink LED1 and LED2 in opposite phases (one ON while other is OFF)
2. Reduce the blink period to 100ms — observe the LED blur effect
3. Add a button (SW0) to toggle the blink on/off using `gpio_pin_get_dt()`
4. Use `k_uptime_get_32()` to print elapsed time every 5 blinks

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day02
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day02/main.cpp](#)